CH-5 PART - II

In-Depth Study Notes: Oracle SQL Aggregation, Grouping, and Filtering

As an expert Oracle SQL Developer, mastering data aggregation and the strict order of execution within the Oracle Engine is critical for reporting and business intelligence. Based on the execution logs provided (CH5-2.TXT), these notes cover **Group Functions**, the **GROUP BY** clause, and the strict architectural differences between the **WHERE** and **HAVING** clauses. Below is a detailed technical breakdown of these core concepts.

Concept 1: Core Summary (Aggregate) Functions

Core Purpose: Summary functions evaluate a multi-row dataset and return a single, aggregated scalar value 1. If no grouping is applied, the function condenses the entire filtered table into a single output row.

Keyword Breakdown:

- **MAX(column) / MIN(column)**: Identifies the highest or lowest value within a dataset. *Why use them?* They are universally compatible with Number, Character, and Date data types 1.
- **SUM(column) / AVG(column)**: Calculates the total or mathematical average. *Constraint:* These strictly require numeric data types 2, 3.
- **COUNT(*) / COUNT(column)**: Evaluates the number of rows.

Syntax Breakdown:

```
SELECT aggregate_function(column_name)
FROM table_name
WHERE condition;
```

Detailed Example: To find the absolute highest salary currently paid within a specific department (e.g., Department 110) in the HR schema:

```
SELECT MAX(SALARY)
FROM EMPLOYEES
WHERE DEPARTMENT_ID = 110;
```

Execution Logic:

1. **FROM**: The engine loads the EMPLOYEES table into the buffer.
2. **WHERE**: It filters out any row where the DEPARTMENT_ID is not 110.
3. **Aggregation Phase**: It scans the remaining rows to find the maximum SALARY value.
4. **SELECT**: It projects that single numeric value to the screen.

Pro-Tips/Best Practices:

- **The Single-Group Rule:** If you attempt to SELECT a non-summary column (like DEPARTMENT_ID) alongside an aggregate function (like MAX(SALARY)) without using a GROUP BY clause, the Oracle engine will immediately halt execution and throw an **ORA-00937: not a single-group group function** error 4.

Concept 2: The GROUP BY Clause

Core Purpose: The GROUP BY clause divides the result set into distinct sub-groups or buckets based on identical values in a specified column or expression. The Oracle engine then applies the summary function independently to each bucket 5, 6.

Keyword Breakdown:

- **GROUP BY**: Instructs the database to aggregate data based on shared properties. It supports pure/original column names and complex non-summary expressions 1.

Syntax Breakdown:

```
SELECT non_aggregate_column, aggregate_function(column_name)
FROM table_name
GROUP BY non_aggregate_column;
```

Detailed Example: To generate a payroll report showing the absolute lowest salary paid per specific Job Title:

```
SELECT JOB_ID, MIN(SALARY)
FROM EMPLOYEES
GROUP BY JOB_ID
ORDER BY 1;
```

Execution groups the employees into 12 distinct buckets (e.g., IT_PROG, SA_REP) and calculates the minimum salary for each 5, 6.

Execution Logic:

1. **FROM**: The engine loads the table.
2. **GROUP BY**: The engine hashes the data into separate virtual buckets based on distinct JOB_ID values.
3. **Aggregation**: The MIN(SALARY) calculates the lowest value locally within each bucket.
4. **SELECT**: Projects the grouped identifiers and their corresponding aggregates.

Pro-Tips/Best Practices:

- **Strict SELECT Constraints:** The golden rule of aggregation is that **any non-summary item listed in the SELECT clause MUST be included in the GROUP BY clause** 1, 4.

Concept 3: Data Filtering (WHERE vs. HAVING)

Core Purpose: While both clauses filter data, their architectural placement in the Oracle Engine's execution path dictates *what* they can filter. WHERE filters raw data *before* grouping, while HAVING filters aggregated data *after* grouping 4.

Keyword Breakdown:

- **WHERE**: Row-level filter. It supports **only non-summary items** and evaluates original table conditions 4, 7.
- **HAVING**: Group-level filter. It supports **all summary items** (like SUM and AVG) plus any non-summary items explicitly defined in the GROUP BY clause 4.

Syntax Breakdown:

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE row_level_condition
GROUP BY column_name
```

```
HAVING group_level_condition;
```
Comparison of Filtering Clauses:
Feature,WHERE Clause,HAVING Clause
Execution Timing,Before GROUP BY 4.,After GROUP BY 4.
Supports Aggregate Functions?,NO. Triggers ORA-00934 4.,YES. Evaluates aggregates 4.
Filters...,Individual Rows 4.,Grouped Summary Rows 4.
Detailed Example: To display the total salary paid per Manager, but *only* for managers whose team's total combined salary exceeds $20,000:
```
SELECT MANAGER_ID, SUM(SALARY)
FROM EMPLOYEES
GROUP BY MANAGER_ID
HAVING SUM(SALARY) > 20000;
```
Execution Logic (Order of Execution Precedence): As explicitly defined by Oracle architecture, the strict execution precedence is: **FROM -> WHERE -> GROUP BY -> HAVING -> ORDER BY** 4.

1. **FROM**: Loads data.
2. **WHERE**: Discards non-matching raw rows (if present).
3. **GROUP BY**: Buckets the surviving rows by MANAGER_ID.
4. **HAVING**: Evaluates the newly calculated SUM(SALARY). If a manager's bucket is $\le$ 20000, that entire bucket is discarded 4, 8.

Pro-Tips/Best Practices:

- **Avoid ORA-00934:** Never put a group function inside a WHERE clause. Attempting to execute WHERE SUM(SALARY) > 20000 forces the engine to halt immediately and throw an **ORA-00934: group function is not allowed here** error 4.
- **Performance Optimization:** While HAVING can filter non-summary items (e.g., HAVING JOB_ID NOT IN ('ST_CLERK')), it is highly inefficient. Always filter raw rows out using the WHERE clause *first* (e.g., WHERE JOB_ID NOT IN ('ST_CLERK')) so the database doesn't waste memory grouping records it will ultimately discard 9-11.

Concept 4: Advanced Expression-Based Grouping

Core Purpose: You are not limited to grouping by standard column names. You can use single-row functions to dynamically manipulate the data into custom buckets prior to aggregation.
Keyword Breakdown:

- **Single-Row Functions (TO_CHAR, SUBSTR, CASE)**: Used to transform the raw column data into a new format, which the GROUP BY clause then evaluates 2, 12.

Syntax Breakdown:
```
SELECT function(column_name), aggregate_function(column_name)
FROM table_name
GROUP BY function(column_name);
```
Detailed Example 1 (Temporal Grouping): To calculate the total salary paid based strictly on the *Quarter* the employees were hired:
```
SELECT TO_CHAR(HIRE_DATE, 'Q'), SUM(SALARY)
FROM EMPLOYEES
```

```
GROUP BY TO_CHAR(HIRE_DATE, 'Q');
```
Execution dynamically calculates the quarter (1, 2, 3, 4) for every row, groups them into 4 distinct buckets, and sums the salaries 2, 12.

Detailed Example 2 (Conditional Grouping): To calculate minimum and maximum salaries grouped dynamically by whether the hire quarter is mathematically 'EVEN' or 'ODD':

```
SELECT CASE WHEN MOD(TO_CHAR(HIRE_DATE,'Q'), 2) = 0 THEN 'EVEN' ELSE
'ODD' END QTR,
       MIN(SALARY),
       MAX(SALARY)
FROM EMPLOYEES
GROUP BY CASE WHEN MOD(TO_CHAR(HIRE_DATE,'Q'), 2) = 0 THEN 'EVEN'
ELSE 'ODD' END;
```
Execution outputs two distinct buckets: 'EVEN' and 'ODD' 12, 13.

Execution Logic:

1. The engine performs the mathematical or string manipulation row-by-row in memory.
2. It groups the rows matching the exact output of those manipulated strings.
3. It applies the aggregate function per custom bucket.

Pro-Tips/Best Practices:

- **Exact Matching:** The expression you write in the SELECT list must perfectly, character-for-character, match the expression written in the GROUP BY clause, otherwise the engine will fail to recognize the grouping logic 2, 12.